

Hardware-Software codesign

Final project report

Group 15

Authors:

Name: Otto Nordenbæk

AU ID: AU764968

Student ID: 202406052

Name: Magnus Ingemansen

AU ID: 772184

Student ID: 202405974

Instructor:

Jalil Boudjadar

Contents

1	Introduction.....	2
1.1	Introduction to requirements	2
2	Solution	3
2.1	Introduction to architecture and decisions.....	3
2.2	Use Case View	3
2.3	Logical View	5
	2.3.1 Sequence diagram(s)	5
	2.3.2 State Diagram(s)	6
2.4	Implementation View.....	11
	2.4.1 SW implementation.....	11
	2.4.2 Hardware design.....	12
	2.4.3 System Synthesis (Integration of SW IP and HW)	13
	2.4.4 Performance analysis	14
3	Discussion of results.....	16
4	Conclusion	17

1 Introduction

In this project, we developed a complete alarm system using the hardware/software co-design methodology from the course. Starting with a clear functional specification, we modelled behaviour using state machines and partitioned the system between software on the Zynq Processing System (PS) and hardware-accelerated logic in the Programmable Logic (PL).

Following the co-design workflow—specification, modelling, optimization, synthesis, and integration—we built a system that continuously monitors sensor data and camera frames, responding in real time to intrusions.

When armed, the system collects data from two sensors and a 9×9 camera frame. Intrusion detection involves combining sensor values with each of the 81 pixels, resulting in 81 multiplications per frame. This computation is accelerated in hardware using Vivado HLS, while control and state logic run efficiently in software on the PS.

The design reflects the core idea of co-design: using hardware for parallel, latency-critical tasks and software for flexible control and coordination.

1.1 Introduction to requirements

The alarm system adheres precisely to the functional requirements outlined in the project specification:

- The system begins in the *Inactive* state and transitions to the *Active* state only after the user enters a valid PIN code. A PIN is considered valid if it is an even integer.
- In the *Active* state:
 - Each sensor generates a random value between 0 and 5 every 60 milliseconds.
 - The camera outputs a 9×9 matrix of values ranging from 0 to 9 every 120 milliseconds.
- The camera produces a 9×9 matrix of values between 0 and 9 every 120 ms.
- For every camera frame, the intrusion score is calculated using the formula:
$$(S1 + S2) \times C[i,j]$$

This is done for all 81 pixels. The total score is the sum of these 81 products. If the total score is **greater than or equal to 1900**, an intrusion is detected.
- Upon detecting an intrusion, the system enters the **Alarmed** state. It remains in this state until either:
 - A new valid PIN code is entered, or
 - 10 seconds have elapsed.

- Once the alarm is cleared, the system returns to the **Active** state and resumes monitoring.

These functional rules form the foundation for the system's state machine models, software modules, and the subsequent synthesis of the hardware accelerator. They also establish the timing constraints and dataflow patterns that guide the hardware/software partitioning strategy.

2 Solution

2.1 Introduction to architecture and decisions

The architecture of the alarm system follows a typical HW/SW co-design structure, with a clear division between the Processing System (PS) and the Programmable Logic (PL). Control-oriented, sequential tasks were handled in software, while the data-parallel intrusion calculation was offloaded to hardware using Vivado HLS.

On the **PS side**, we implemented the system controller, user input handling (PIN code), state machines for system states, and timing for sensor and camera sampling. These tasks are event-driven and suited for flexible software execution. The PS also manages the hardware accelerator by writing inputs, starting the computation, and reading the result via AXI.

The PL handles the intrusion score calculation, which multiplies each pixel in the 9×9 camera frame by (S1 + S2) and sums the results. Since this operation is repetitive and data-parallel, it was ideal for acceleration. Using Vivado HLS, we converted the C implementation into an IP core with AXI interfaces and applied optimizations like pipelining and loop unrolling during design space exploration.

Communication between PS and PL uses an **AXI-Lite interface**, which is sufficient given the system's low data rate. The PS writes inputs to the accelerator, triggers it, and waits for the output.

In summary, the architecture reflects core co-design principles: flexible control in software, efficient computation in hardware, and simple, synchronized communication through AXI.

2.2 Use Case View

The Use Case View describes how the alarm system functions from the user's perspective, emphasizing system behaviour in various scenarios. It focuses solely on user-system interaction, without detailing hardware or implementation specifics.

Actors:

- **User:** Enters the pin code and acknowledges alarms.
- **Alarm System:** Monitors the environment, detects intrusions, and manages system states.

Use Case 1: Arm System (Enter Pin Code)

The user initiates activation by entering a PIN code. The system checks whether the PIN is valid - defined as an even integer.

- If valid, the system transitions from the *Inactive* state to the *Active* state
- If invalid, the system remains *Inactive*.

This is the only method by which the user can activate the system.

Use Case 2: Monitor Environment

Once active, the system automatically begins monitoring the environment.

- The sensors generate values every 60 milliseconds.
- The camera produces a 9×9 frame every 120 milliseconds.

There is no user interaction in this phase—the system continuously collects and processes data in the background.

Use Case 3: Detect Intrusion

For each camera frame, the system calculates an intrusion score by computing:

$(S1 + S2) \times C[i,j]$ for all 81 pixels.

- The sum of these 81 products is the total intrusion score.
- If the score is greater than or equal to 1900, the system detects an intrusion.
- Upon detection, the system enters the *Alarmed* state and activates a distinct LED pattern to indicate that an alarm has been triggered.

Use Case 4: Acknowledge Alarm

While in the *Alarmed* state, the user can enter a new PIN code:

- If the PIN is valid, the alarm is acknowledged, and the system returns to the *Active* state to resume monitoring.
- If the user takes no action, the system automatically resets after 10 seconds, returning to the *Active* state.

The LED indicators update to reflect the current state of the system.

Summary

These use cases summarize the full interaction between the user and the alarm system. The user only enters PIN codes to arm or disarm; all monitoring, detection, and state transitions are handled automatically.

2.3 Logical View

2.3.1 Sequence diagram(s)

The following is a **UML sequence-style diagram** that illustrates the interaction between the system's primary components:

- Sensors S1 and S2
- Camera Module
- Alarm Finite State Machine (FSM)
- Intrusion Calculation Block
- 10-Second Timer
- Alarm Output

The sequence begins when the user enters a valid PIN code, which triggers a transition in the system's state—from *Inactive* to *Active*, and later to *Alarmed*, depending on intrusion detection events.

Each component plays a distinct role in this interaction:

- Sensors S1 and S2: Generate environmental readings every 60 milliseconds.
- Camera Module: Captures a 9×9 frame every 120 milliseconds.
- Intrusion Calculation Block: Computes the intrusion score using sensor values and camera data.
- Alarm FSM: Manages the system states and controls transitions based on input conditions and results from the intrusion calculation.
- 10-Second Timer: Starts when the system enters the *Alarmed* state and returns the system to *Active* if the user does not acknowledge the alarm.
- Alarm Output: Visually signals an intrusion using LEDs or another output mechanism.

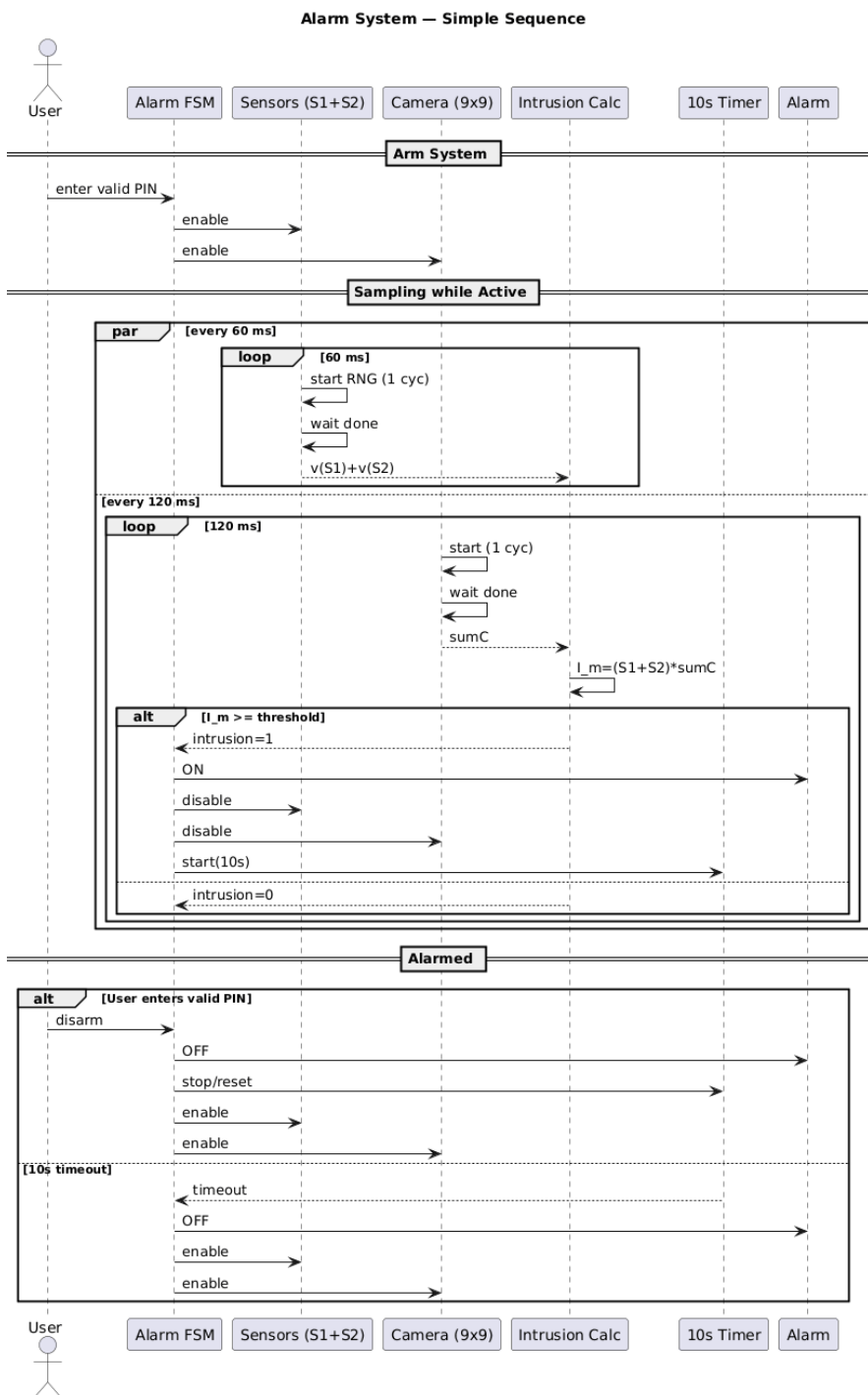


Figure 1 sequence diagram of system

2.3.2 State Diagram(s)

State machines for the individual elements (sensor, camera), and then the state machine of the entire system

The following section describes the state machines for the two sensors, the camera module, and the overall system behavior. Each state machine outlines the states, transitions, and timing logic that govern how each component operates.

Sensor State Machine

This state machine defines the behavior of Sensor 1 (S1) and Sensor 2 (S2). Each sensor operates in a timed cycle, collecting data every 60 milliseconds.

States:

- **S_IDLE:**
The initial state. The sensor is inactive and not yet triggered to begin operation.
- **S_START:**
The sensor is activated and prepares to begin data collection.
- **S_WAIT_DONE:**
The sensor is actively collecting data during this state.
- **S_WAIT_60MS:**
After data collection, the sensor processes the data and waits 60 ms before restarting the cycle.

Transitions:

- **S_IDLE → S_START:**
Triggered when the system enters the Active state.
- **S_START → S_WAIT_DONE:**
Initiates the data acquisition process.
- **S_WAIT_DONE → S_WAIT_60MS:**
Upon completion of data collection.
- **S_WAIT_60MS → S_START:**
After 60 ms delay, the sensor begins a new cycle of data collection.

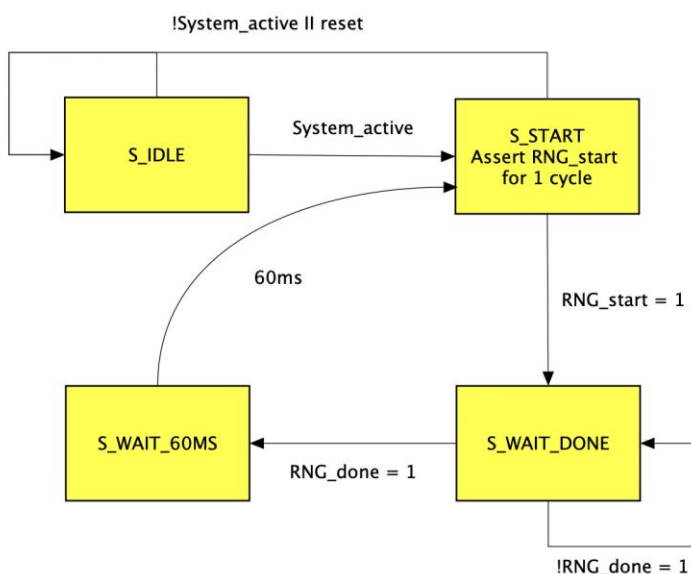


Figure 2 state machine diagram of sensor

Camera State Machine

The camera follows a similar cycle to the sensors, but it operates on a 120 ms interval and collects a 9×9 frame each cycle.

States:

- **S_IDLE:**
The camera is inactive and awaiting system activation.
- **S_START:**
The camera is activated and prepares to begin frame capture.
- **S_WAIT_DONE:**
The camera is capturing the 9×9 image data.
- **S_WAIT_120MS:**
After capturing, the system processes the frame and waits 120 ms before starting the next cycle.

Transitions:

- **S_IDLE → S_START:**
Triggered when the system transitions to the Active state.
- **S_START → S_WAIT_DONE:**
Begins frame acquisition.
- **S_WAIT_DONE → S_WAIT_120MS:**
Transitions after completing the capture.
- **S_WAIT_120MS → S_START:**
After a 120 ms wait, restarts the frame collection process.

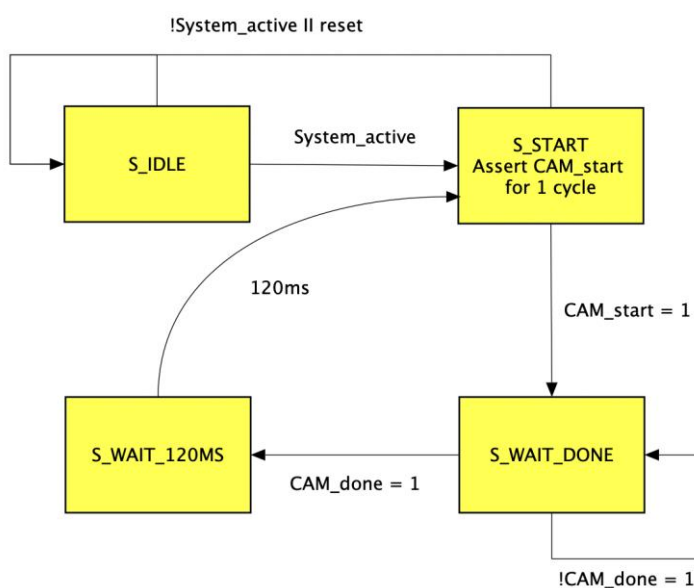


Figure 3 state machine diagram of camera

System State Machine

The system-level state machine governs the overall behaviour of the alarm system, transitioning between *Inactive*, *Active*, and *Alarmed* states.

States:

- *Inactive*:
The system is idle. Sensors and camera are inactive. Awaiting valid PIN input.
- *Active*:
The system actively monitors the environment using sensors and the camera.
- *Alarmed*:
An intrusion has been detected. Alarm is triggered and a timer is started.

Transitions:

- Inactive → Active:
Triggered when the user enters a valid PIN (even number).
- Active → Alarmed:
Occurs when the intrusion score ≥ 1900 is detected by the intrusion calculation block.
- Alarmed → Active:
Two possible transitions:
 - Valid PIN is entered by the user to acknowledge the alarm.
 - 10-second timer expires without user interaction.

In both cases, the system resets to the Active state and resumes monitoring

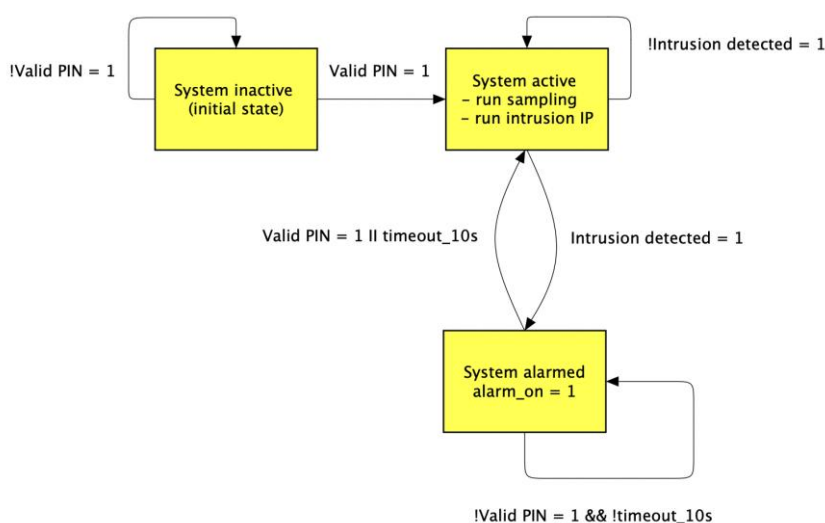


Figure 4 state machine diagram of entire system

2.4 Implementation View

2.4.1 SW implementation

- HLS for SW (RTL IP core generation)
- Design Space exploration for SW

The Implementation View shows how software and hardware responsibilities were divided using a HW/SW co-design approach.

The software runs on the Zynq Processing System (PS) and handles control tasks such as PIN entry, state machine management, sensor and camera sampling, data transfer to the hardware accelerator, and response handling. It is organized around three switch-based state machines—for sensors, the camera, and the alarm controller.

Environmental inputs are simulated: every 60 ms, the software generates two random sensor values (0–5); every 120 ms, it generates a 9×9 matrix with pixel values (0–9) to represent camera frames.

When data is ready, the software writes it to the hardware accelerator via AXI-Lite registers, triggers the computation, and waits for completion. The result (intrusion score) is then read back from the output register.

The hardware accelerator, developed in Vivado HLS, converts a C function into an RTL IP core. Optimization directives like pipelining and loop unrolling were applied to improve performance. The computation multiplies each pixel by $(S1 + S2)$ and sums the results.

If the score is 1900 or more, the software transitions the system to the Alarmed state and activates an LED pattern via GPIO. The alarm clears when the user enters a valid PIN or after a 10-second timeout.

In short, the software manages control and data flow, while the hardware performs fast intrusion detection—achieving an efficient and modular HW/SW co-design.

2.4.2 Hardware design

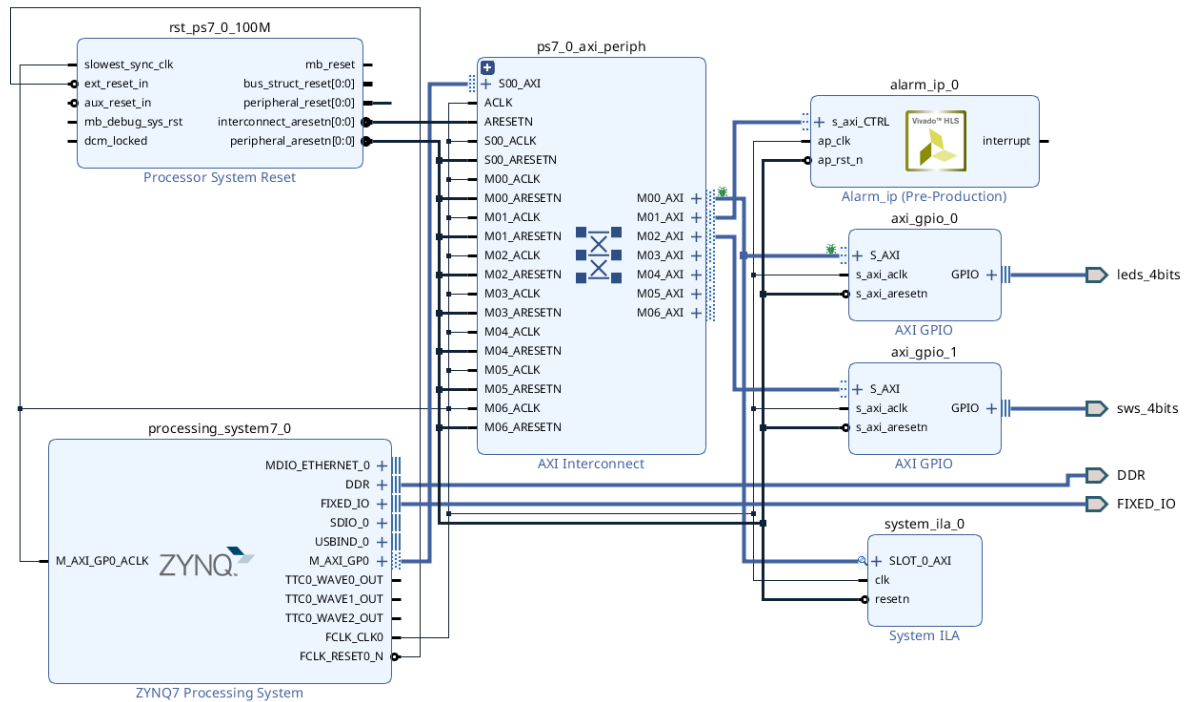


Figure 5 hardware design overview

The hardware design was developed in Vivado using the Zynq-7000 Processing System along with a minimal set of AXI-connected peripherals. The primary goal was to integrate the HLS-generated intrusion detection module (`alarm_ip_0`) into the programmable logic and enable smooth communication with the software on the ARM cores. The design structure followed the same methodology used in the course labs.

The core of the system is the ZYNQ7 Processing System, which provides the ARM processors, memory controller, clocks, and the `M_AXI_GP0` interface. This AXI master port allows the software to access peripherals in the programmable logic using memory-mapped reads and writes.

The `M_AXI_GP0` connects to an AXI Interconnect, which handles address routing and decoding. This setup allows easy integration of multiple peripherals without custom wiring.

The intrusion detection IP, `alarm_ip_0`, is connected as an AXI-Lite slave. The processor writes sensor and camera data to its registers, starts the computation, and reads back the intrusion result. The PL subsystem runs on the `FCLK_CLK0` clock from the PS, with reset signals handled by the Processor System Reset block for synchronization.

Two AXI GPIO peripherals were added—one for controlling onboard LEDs to indicate system state, and another connected to the switches for simple input. This setup provided easy, real-time feedback without the need for external tools.

Vivado may insert a System ILA block during development, but in this case, it was unused and did not affect the final design.

In conclusion, the hardware setup is simple and efficient: the Processing System handles control, the programmable logic performs acceleration, and AXI-Lite enables seamless communication. This design clearly follows the hardware/software co-design principles taught in the course.

HLS for SW (RTL IP core generation)

The intrusion calculation was first implemented as a C function that takes two sensor inputs (S1, S2) and a 9×9 matrix, returning the total intrusion sum. This function was set as the top-level function in a Vivado HLS project, where we added the source code, a simple testbench, and configured the clock and device to match the Zynq platform.

To enable communication with the Processing System (PS), all function arguments were mapped to an AXI-Lite interface. This allowed the PS to write inputs and read the output through memory-mapped registers. We verified the design through C simulation, followed by C synthesis and C/RTL co-simulation to confirm correctness of the RTL implementation.

After confirming the design met the target performance of 92 clock cycles or less, we exported the RTL as an IP core. This IP was then integrated into the Vivado block design and connected to the PS via AXI, finalizing the hardware acceleration setup.

Design Space exploration for SW

We performed a brief design space exploration of the intrusion calculation in Vivado HLS to assess how optimization directives affect latency and resource usage.

The baseline implementation was a direct C version without optimizations, serving as a reference for both performance and area. In the optimized version, we applied loop pipelining and partial unrolling to the 81-pixel loop, which significantly reduced latency and improved throughput, at the cost of increased LUT and DSP usage.

Vivado HLS reports were used to compare the two designs, focusing on latency, initiation interval, and resource utilization (LUTs, flip-flops, BRAMs, DSPs).

These results were later used in the performance analysis to evaluate trade-offs and determine the implementation that best balanced speed and resource efficiency for our system.

2.4.3 System Synthesis (Integration of SW IP and HW)

After generating the HLS IP core, it was integrated into the Vivado block design and connected to the Zynq Processing System via an AXI-Lite interface. An AXI Interconnect managed address mapping, allowing the processor to interact with the IP core through standard memory-mapped registers for reading and writing data.

The IP core and GPIO peripherals (for LEDs and switches) were assigned address ranges using the Vivado Address Editor. All components shared the FCLK_CLK0 clock and received reset signals from the Processor System Reset block for synchronized initialization.

Once the hardware setup was complete, the platform was exported to Vitis. The software project used auto-generated drivers for `alarm_ip_0`, enabling simple control of the hardware accelerator without manual register handling.

After programming the FPGA, the software ran on the ARM processor. The system was now fully integrated: the Processing System managed control and data flow, while the Programmable Logic performed intrusion detection. GPIO-controlled LEDs confirmed system behaviour and alarm status.

This final integration step completed the co-design, combining efficient hardware acceleration with software-driven control in a unified embedded system.

2.4.4 Performance analysis

To evaluate the final system, we compared two HLS implementations of the intrusion computation and analyzed their impact on overall system performance. The two versions were:

- A baseline design, using a straightforward C implementation without any optimization directives.
- An optimized design, where HLS directives such as loop pipelining and partial unrolling were applied to the 81-element computation loop. All three implementations use the same algorithmic structure; the differences arise purely from HLS directive configurations.

The analysis focused on latency, resource usage, and system responsiveness on the Zynq platform. The baseline version had significantly higher latency due to its sequential execution of multiplications. This led to longer wait times for each frame. In contrast, the optimized version overlapped operations using pipelining, reducing the initiation interval and total computation time.

As expected, the optimized design consumed more LUTs and DSPs because of increased parallelism. However, this trade-off is consistent with design space exploration principles. The design still fit comfortably within the Zynq-7000's resource limits.

Once integrated, the optimized version noticeably improved system responsiveness. Intrusion detection was faster, LEDs reacted immediately, and the Processing System spent less time waiting—resulting in a smoother monitoring cycle.

In summary, the optimized HLS design reduced latency and improved throughput without exceeding hardware constraints. This validates the effectiveness of hardware acceleration and design space exploration in achieving a balanced co-design.

Below is a table constructed of the different versions: Default rolled, Unrolled and Default Pipelined rolled where we compare their loop directive, latency (cycles), time (ns), FF and LUT.

Version	Loop Directive	Latency (cycles)	Time (ns)	FF	LUT
Default rolled	none	250	$5 \cdot 250 = 1250$	693	1325
Unrolled	Unroll factor = 9	134	$134 \cdot 5 = 670$	796	1714
Default Pipelined rolled	Pipeline II = 1	90	$90 \cdot 5 = 450$	690	1344

Then we've made a visualisation of these comparisons in a single dual-axis graph where we can clearly see which versions outperform each other in what ways. We chose dual-axis because they vary on different numeric scales and thus plotting them on the same y-axis would either flatten one group or exaggerate the other group, a single plot avoids 4 separate graphs and provides immediate comparative clarity and comparing latency and LUT visually gives insight into the HW/SW trade-offs.

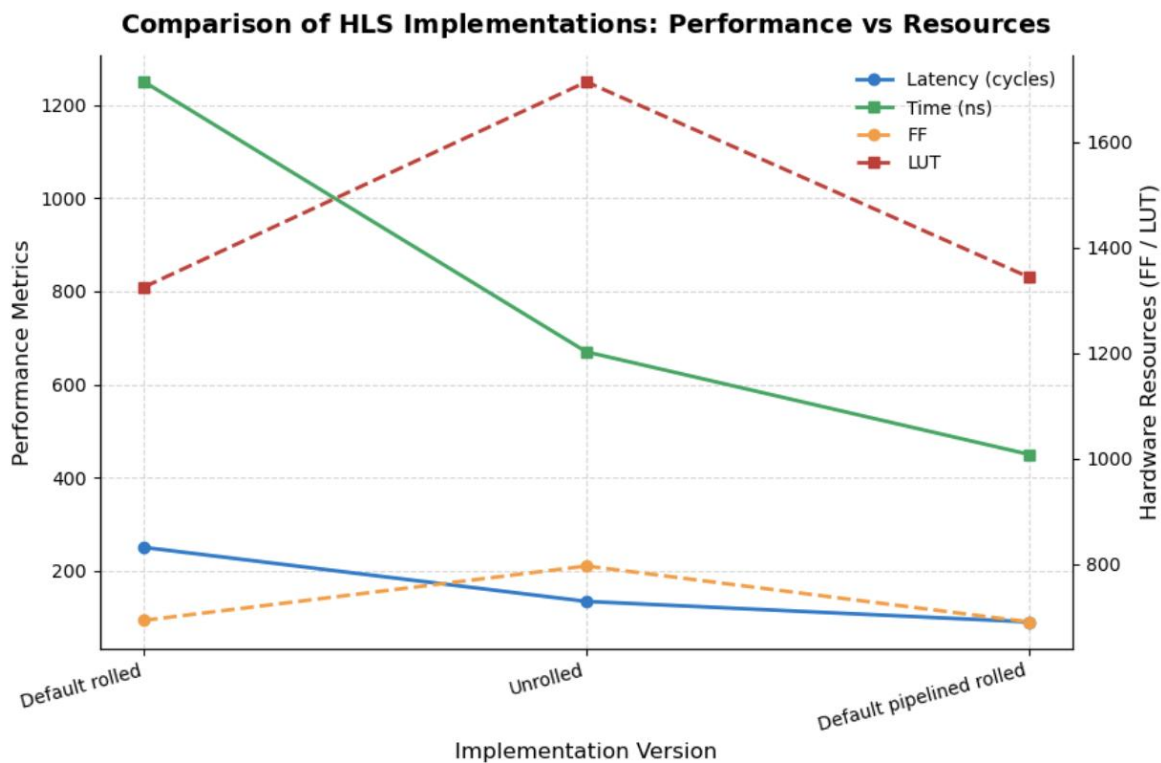


Figure 6 HLS version comparison

3 Discussion of results

To better understand the performance and area trade-offs of the intrusion computation, we explored three versions of the same C loop in Vivado HLS: a default (rolled) version, an unrolled version, and a pipelined version. Each version resulted in a different hardware implementation, demonstrating how various HLS directives can significantly change latency, resource usage, and throughput.

Default (rolled):

- Same HW used each iteration
- Small area (FF=693, LUT=1325)
- Long latency (250 cycles, 1250 ns)

This version corresponds to the baseline design where no optimization directives were applied. Because each iteration must wait for the previous one to finish, the overall latency is high. However, the area usage is minimal, making it suitable for systems with tight resource constraints but less stringent performance requirements

Unrolled:

- More HW instances used in parallel
- Higher area (FF=796, LUT=1714)
- Shorter latency (134 cycles, 670 ns)

In the unrolled version, the loop is partially or fully unrolled, allowing multiple iterations to execute simultaneously. This significantly improves performance but at the cost of increased area usage. More logic is needed to support the parallelism, which could become a limiting factor depending on available FPGA resources.

Pipelined (chosen):

- Iterations executed in a pipeline (overlapped)
- Medium area (FF=690, LUT=1344)
- Short latency (90 cycles, 450 ns)
- Best latency/area trade-off

The pipelined version provided the best trade-off, overlapping loop iterations to reduce latency while keeping area usage close to the baseline. We considered calculating the throughput in GIPS for this version but since throughput is only really viable in pipelined systems and thus only relevant for this version and not either of the two first versions, we chose to leave it out to avoid confusion.

The three implementations show how different HLS directives can yield vastly different hardware from the same C loop. The rolled version minimizes area but is slow; the unrolled version is fast but resource heavy. The pipelined version strikes a balance, meeting latency targets without excessive resource use.

This reflects the core purpose of design space exploration—testing various hardware mappings of the same algorithm to find the most suitable design. For our system, pipelining achieved the best balance of speed and area, making it the optimal choice.

4 Conclusion

This project successfully demonstrates the application of hardware/software co-design to build an efficient real-time alarm system on the Zynq-7000 platform. The software managed control flow, user input, and data coordination, while the hardware—implemented using Vivado HLS—accelerated the intrusion detection.

Design space exploration allowed us to evaluate multiple hardware implementations of the intrusion computation. The pipelined version achieved the best balance between latency and resource usage, meeting the performance targets without exceeding the device constraints. Through AXI-Lite integration and Vitis-based development, we created a tightly coupled HW/SW system that responds quickly when the intrusion threshold is exceeded.

At the same time, the prototype has some clear limitations. The sensors and 9×9 camera frame are modeled using simple random values, the intrusion algorithm is a basic linear sum with a fixed threshold, and the camera resolution is relatively low. These choices make the system easy to implement and test, but they do not fully represent a realistic alarm installation.

In future work, the system could be extended with more realistic input sources (for example real sensors or a camera interface), higher-resolution frames, and more advanced intrusion detection algorithms. Such extensions would move the design closer to a deployable embedded alarm system, while still following the same HW/SW co-design principles used in this project.